

E2E Service System - Multi-Tenant Architecture

1. Multi-Tenant Overview

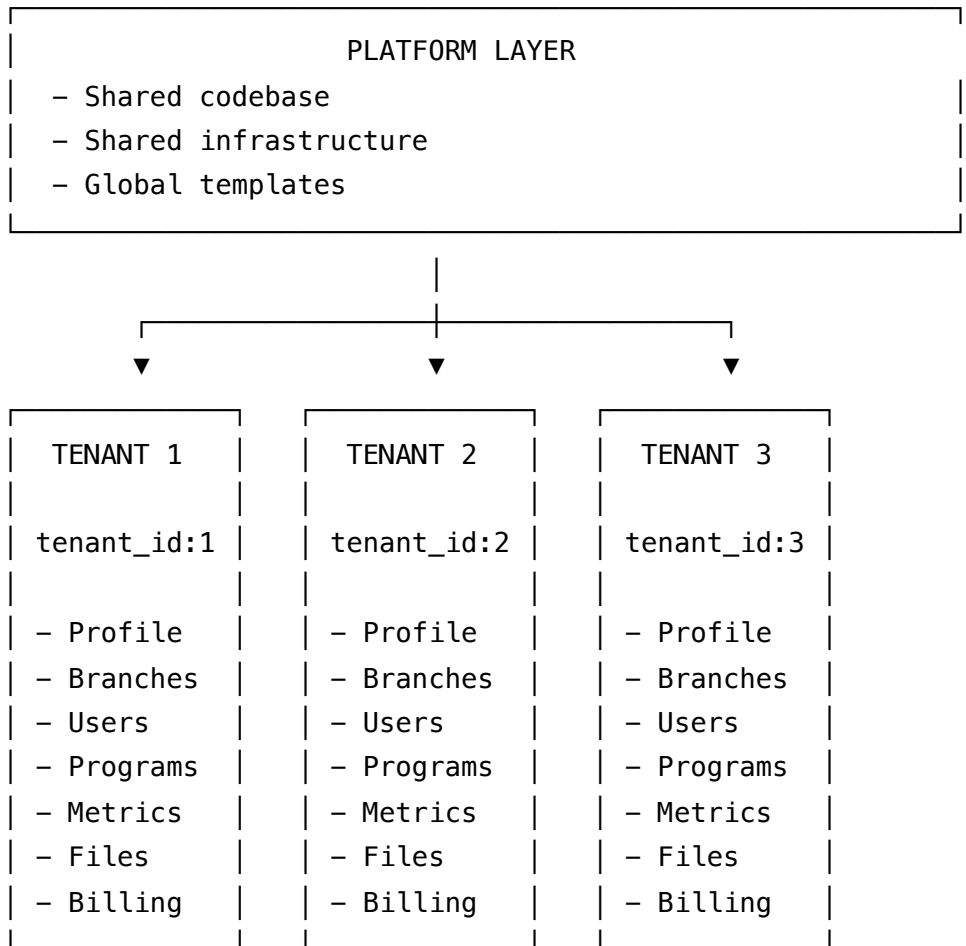
What is Multi-Tenancy?

One platform serves multiple clients (tenants), each with complete data isolation.

Key Principles

- **Data Isolation:** Each tenant's data is completely separate
- **Shared Infrastructure:** All tenants use same codebase and servers
- **Tenant Context:** Every request includes tenant ID (Org ID)
- **Row-Level Security:** Database enforces tenant isolation

2. Tenant Structure



3. Data Isolation Strategy

Option 1: Shared Database with Row-Level Security (Recommended)

Pros:

- Cost-effective (one database)
- Easy to manage
- Good for 10-100 tenants

Cons:

- Risk of data leakage (if not implemented correctly)
- Performance can degrade with many tenants

Implementation:

```
-- Every table has tenant_id column
CREATE TABLE users (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  email VARCHAR(255),
  ...
);

-- Row-level security policy
CREATE POLICY tenant_isolation ON users
  USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

Option 2: Separate Database per Tenant

Pros:

- Complete data isolation
- Better security
- Easier to scale individual tenants

Cons:

- Higher cost (multiple databases)
- Complex to manage
- Harder to run cross-tenant analytics

Option 3: Hybrid (Shared DB + Separate for Large Tenants)

Pros:

- Best of both worlds
- Small tenants share DB, large tenants get dedicated DB

Cons:

- Most complex to implement

4. Tenant Identification

Subdomain-Based

`https://client-a.e2e.com` → Tenant: `client-a`
`https://client-b.e2e.com` → Tenant: `client-b`

Path-Based

`https://e2e.com/client-a` → Tenant: `client-a`
`https://e2e.com/client-b` → Tenant: `client-b`

Header-Based (API)

GET `/api/users`
Headers:
 X-Tenant-ID: `client-a`
 Authorization: Bearer `<token>`

5. Request Flow

[Client Browser] → [Load Balancer] → [API Gateway]

↓

[Tenant Middleware]

- Extract tenant_id from subdomain/header
- Validate tenant exists
- Set tenant context

↓

[Auth Middleware]

- Validate JWT token
- Check user belongs to tenant

↓

[RBAC Middleware]

- Check user role permissions

↓

[Controller]

- Business logic

↓

[Database]

- Query with tenant_id filter
- Row-level security enforced

6. Database Schema (Multi-Tenant)

Core Tables

```
-- Tenants table
CREATE TABLE tenants (
  id UUID PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  subdomain VARCHAR(100) UNIQUE NOT NULL,
  plan VARCHAR(50), -- Starter, Growth, Enterprise
  status VARCHAR(50), -- Active, Suspended, Cancelled
  created_at TIMESTAMP DEFAULT NOW()
);

-- Organizations (one per tenant)
CREATE TABLE organizations (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  name VARCHAR(255),
  industry VARCHAR(100),
  targets JSONB, -- {complaints: -20, csat: 2}
  created_at TIMESTAMP DEFAULT NOW()
);

-- Branches (tenant-scoped)
CREATE TABLE branches (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  org_id UUID NOT NULL REFERENCES organizations(id),
  name VARCHAR(255),
  region VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW()
);

-- Users (tenant-scoped)
CREATE TABLE users (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  org_id UUID NOT NULL,
  branch_id UUID,
  role VARCHAR(50), -- ClientAdmin, Manager, Coach, Staff, ExecutiveViewer
  email VARCHAR(255),
```

```
password_hash VARCHAR(255),  
created_at TIMESTAMP DEFAULT NOW()  
);
```

```
-- All other tables follow same pattern (tenant_id column)
```

Indexes for Performance

```
-- Composite indexes on tenant_id + other columns  
CREATE INDEX idx_users_tenant ON users(tenant_id, email);  
CREATE INDEX idx_branches_tenant ON branches(tenant_id, org_id);  
CREATE INDEX idx_sessions_tenant ON sessions(tenant_id, branch_id);
```

7. Tenant Context Middleware (NestJS)

```
// tenant.middleware.ts
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class TenantMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    // Extract tenant from subdomain
    const host = req.hostname;
    const subdomain = host.split('.')[0];

    // Or extract from header
    const tenantId = req.headers['x-tenant-id'] as string;

    // Validate tenant exists
    const tenant = await this.tenantService.findBySubdomain(subdomain);
    if (!tenant) {
      throw new UnauthorizedException('Invalid tenant');
    }

    // Set tenant context
    req['tenant'] = tenant;
    req['tenantId'] = tenant.id;

    next();
  }
}
```


8. Row-Level Security (Prisma)

```
// prisma.service.ts
import { Injectable } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient {
  async setTenantContext(tenantId: string) {
    await this.$executeRaw`SET app.current_tenant = ${tenantId}`;
  }
}

// Usage in service
async findUsers(tenantId: string) {
  await this.prisma.setTenantContext(tenantId);
  return this.prisma.user.findMany(); // Automatically filtered by tenant_id
}
```

9. File Storage (Multi-Tenant)

S3 Bucket Structure

```
e2e-files/
├── tenant-1/
│   ├── sessions/
│   ├── roleplays/
│   └── coaching/
├── tenant-2/
│   ├── sessions/
│   ├── roleplays/
│   └── coaching/
└── tenant-3/
    └── ...
```

Upload Logic

```
async uploadFile(tenantId: string, file: File) {  
  const key = `${tenantId}/sessions/${Date.now()}-${file.name}`;  
  await s3.upload({  
    Bucket: 'e2e-files',  
    Key: key,  
    Body: file.buffer,  
  });  
  return `https://e2e-files.s3.amazonaws.com/${key}`;  
}
```

10. Billing & Subscriptions

Tenant Plans

Plan	Users	Branches	Price/Month
Starter	1-50	1	\$99
Growth	51-200	5	\$299
Enterprise	200+	Unlimited	Custom

Usage Tracking

```
// Track usage per tenant  
interface TenantUsage {  
  tenantId: string;  
  activeUsers: number;  
  sessionsCreated: number;  
  smsCreditsUsed: number;  
  storageUsedGB: number;  
}
```

11. Tenant Onboarding Flow

```
[Super Admin] → Create Tenant
      ↓
[System] → Generate subdomain (client-a.e2e.com)
      ↓
[System] → Create tenant record in DB
      ↓
[System] → Create default organization
      ↓
[System] → Send welcome email to Client Admin
      ↓
[Client Admin] → Set password, log in
      ↓
[Client Admin] → Import branches/users
      ↓
[Client Admin] → Set 90-day pilot targets
      ↓
[System] → Tenant is active
```

12. Tenant Isolation Testing

Security Tests

```
describe('Tenant Isolation', () => {
  it('should not allow access to other tenant data', async () => {
    // Login as Tenant 1 user
    const token1 = await login('user@tenant1.com');

    // Try to access Tenant 2 data
    const response = await request(app)
      .get('/api/users')
      .set('Authorization', `Bearer ${token1}`)
      .set('X-Tenant-ID', 'tenant-2');

    expect(response.status).toBe(403); // Forbidden
  });
});
```

13. Scaling Strategy

Horizontal Scaling

- Add more API servers (Kubernetes auto-scaling)
- Load balancer distributes requests

Database Scaling

- Read replicas for analytics queries
- Partition large tables by tenant_id
- Move large tenants to dedicated databases

Caching

- Redis cache per tenant
- Cache key format: `tenant:{tenantId}:users`

14. Monitoring & Observability

Per-Tenant Metrics

- Active users per tenant
- API requests per tenant
- Database queries per tenant
- Storage used per tenant
- Error rate per tenant

Alerts

- Tenant exceeds usage limits
- Tenant has high error rate
- Tenant payment failed

15. Tenant Lifecycle

Active

- Full access to platform
- Billing active

Suspended

- Read-only access
- Payment overdue

Cancelled

- No access
- Data retained for 30 days

Deleted

- All data permanently deleted
- Cannot be recovered